

반복 작업을 자동화하는 데서 시작해, 시스템의 책임과 실패 경계를 설계했습니다.

조운호 PLATFORM ENGINEER

홈랩 운영의 반복을 줄이기 위해 자동화를 시작했습니다. 범위가 커지며 생긴 결함과 실패를 겪은 뒤, VM 생성·애플리케이션 배포·사용자 데이터의 수명주기를 분리했습니다.

- 01 반복 운영
- 02 통합 자동화
- 03 책임 결함 발견
- 04 Gjallar·Heimdall 분리
- 05 실패 시 기존 상태 보존

새 서비스를 올릴 때마다, 같은 실행 환경을 다시 만들었습니다.

홈랩에 프로그램을 올리는 일은 애플리케이션 배포만으로 끝나지 않았습니다.
VM과 네트워크를 준비하고 Docker와 내부 실행 환경을 구성한 뒤에야 배포를
시작할 수 있었습니다.

프로젝트가 늘수록 같은 작업을 반복했고, 설정은 조금씩 달라졌습니다. 문제가
생기면 애플리케이션, 네트워크, 실행 환경 중 어디서 실패했는지 다시 추적해야
했습니다.

처음의 목표는 거대한 플랫폼이 아니라, 매번 반복하던 준비 작업을 줄이는
것이었습니다.



←
새 프로젝트가 생기면 처음부터 다시 시작

자연어 요청을 바로 실행하지 않고, 실행 가능한 계획으로 바꿨습니다.

K-PaaS

K-Le-PaaS

NCP Connected

Dashboard

Commands

Deployments

GitHub

Monitoring

Settings

Connected Repositories

K-Le-PaaS/test01

main · 5h ago

K-Le-PaaS/test03

main · 5h ago

K-Le-PaaS/test02

main · 5h ago

View All

Pull Requests

[REFACTOR] 자연어 명령 응답 구조 개선

by @khyeon0620 · 2m ago

[FEAT] 로그 조회 기능 구현

by @khyeon0620 · 6m ago

[FIX] 빌드 에러 해결

by @khyeon0620 · 12m ago

View All

System Utilization

CPU Usage

2.61%

Memory Usage

39.66%

View Details

Alerts

[Demo] High CPU Utilization

Warning

[Demo] High Disk Usage (/)

Warning

[Demo] High Memory Usage

Warning

View All

Active Deployments

Latest repository workloads

View All

K-Le-PaaS/test01

K-Le-PaaS/test01:55225...

2/2 pods ready · 16h ago

Running

K-Le-PaaS/test03

K-Le-PaaS/test03:fa9a5...

1/1 pods ready · 13h ago

Running

K-Le-PaaS/test02

No deployments

No Deploy

K-Le-PaaS/test04

No deployments

No Deploy

Recent Commands

Latest natural language command executions

Start Chat

파드 목록 보여줘

user_message · 8h ago

Success

전체적인 상황 보여줘

user_message · 8h ago

Success

접속 주소 목록 보여줘

user_message · 13h ago

Success

파드 목록 보여줘

user_message · 13h ago

Success

파드 목록 보여줘

user_message · 13h ago

Success

NCP KBs Automation Platform

저장소 검증

K-Le-PaaS · Previous · 2인 팀 프로젝트

자연어 요청은 그대로 실행할 수 있는 입력이 아니었습니다. 의도와 대상을 해석하고 실행 가능한 CommandPlan으로 정규화한 뒤 Kubernetes·NCP 작업에 연결했습니다.

개인 기여

Gemini 의도·엔티티 해석, CommandPlan 경계, Kubernetes 명령 계획, 도메인 동기화와 NKS 모니터링

01

Natural Language

→

02

Intent & Entity

→

03

CommandPlan

→

04

Kubernetes / NCP

→

05

Feedback

입력과 실행 사이에 계획과 확인 단계를 두는 관점이 흐름 자동화의 출발점이 되었습니다.

YUNHO CHO · PLATFORM ENGINEER

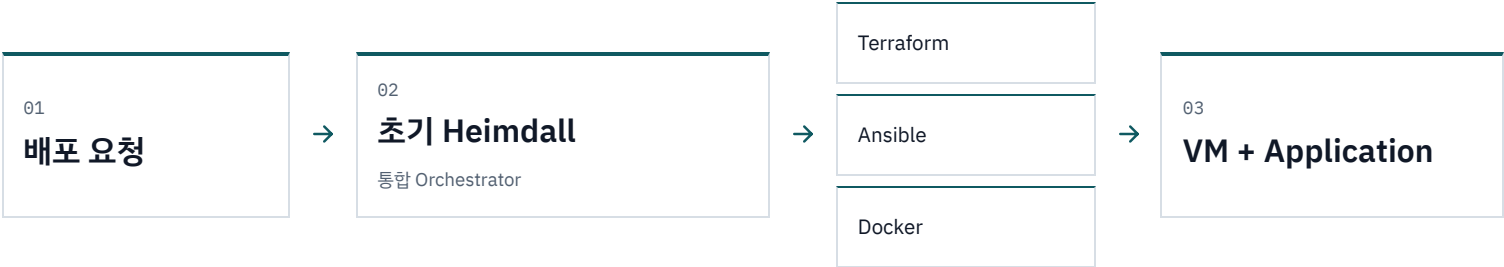
03 / 14

처음에는 VM 생성부터 배포까지 하나의 시스템이 맡아야 효율적이라고 생각했습니다.

반복 작업을 한 번의 요청으로 줄이기 위해 초기 Heimdall은 Terraform 기반 VM 생성, Ansible 기반 설정, 네트워크 구성과 Docker 배포를 하나의 흐름에서 처리했습니다.

실행 경로가 하나면 상태를 추적하기 쉽고 새 서비스를 더 빠르게 올릴 수 있다고 판단했습니다.

- 한 번의 요청
- 반복 감소
- 한 경로에서 추적



자동화가 늘수록, 한 번의 변경이 더 많은 책임을 건드렸습니다.

VM 생성과 애플리케이션 배포는 변경되는 이유가 달랐습니다. 인프라 용량과 네트워크를 바꾸는 일, 애플리케이션 코드와 설정을 배포하는 일이 같은 실행 경로에서 실패했습니다.

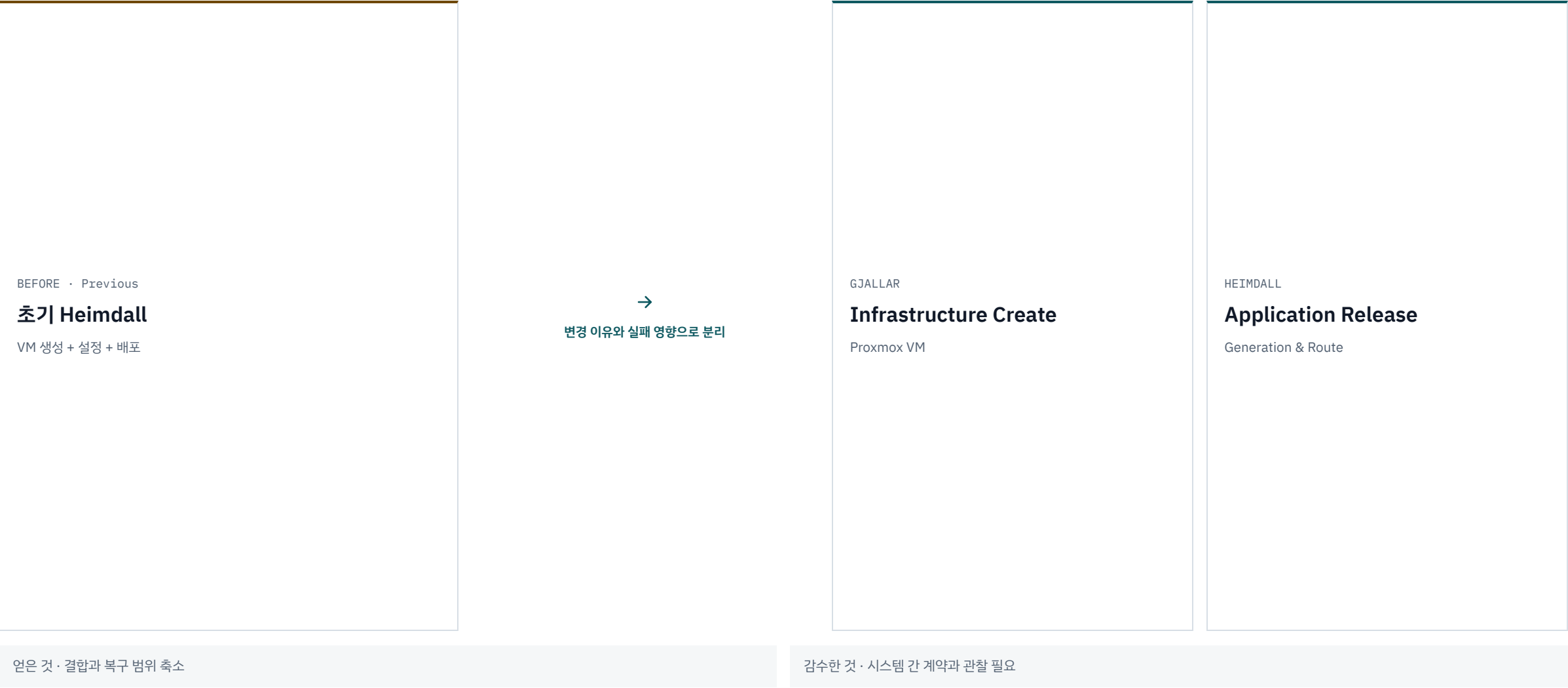
Terraform·Ansible·Docker 의존성을 함께 관리하면서 기능을 추가할수록 책임의 소유자가 흐려졌습니다.

책임 결합	인프라와 배포가 함께 변경
실행 경로 확대	의존성과 실패 지점 증가
복구 범위 확대	서로 다른 실패를 함께 복구

기존 구조 확장	전체 재작성	책임 경계를 다시 정하기
----------	--------	---------------

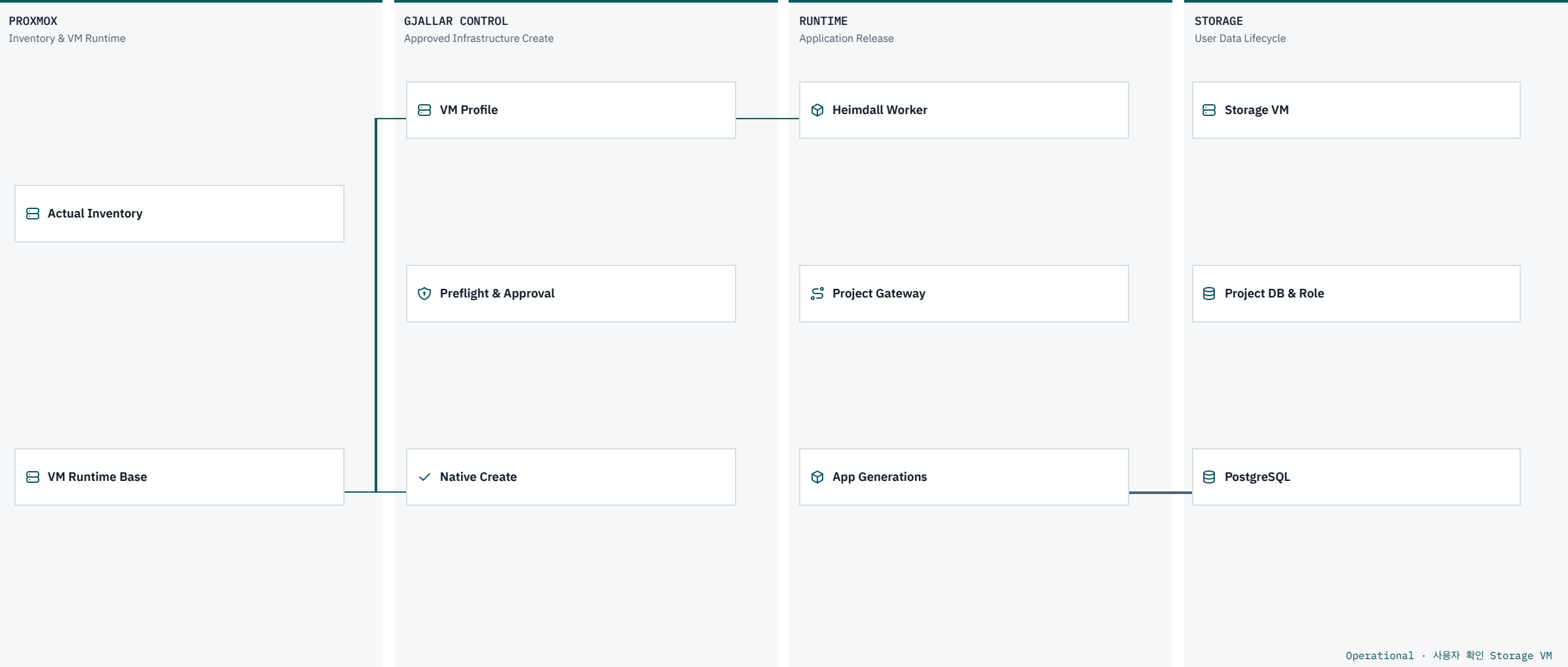
기능을 더 붙이는 대신, 변경 이유가 다른 책임을 두 시스템으로 나눴습니다.

복잡성이 사라진 것은 아닙니다. 두 시스템 사이의 계약과 상태 관찰이 필요해졌지만, 실패했을 때 무엇을 멈추고 무엇을 보존해야 하는지는 더 명확해졌습니다.



실행 기반, 배포 세대, 사용자 데이터의 수명주기를 서로 다르게 다룹니다.

Gjallar는 Proxmox의 실제 상태와 승인된 VM 생성을, Heimdall은 고정한 commit의 배포 후보와 승격을 말합니다. 애플리케이션 실행 세대와 PostgreSQL 사용자 데이터는 같은 실패 정책으로 다루지 않습니다.



Operational · 사용자 확인 Storage VM

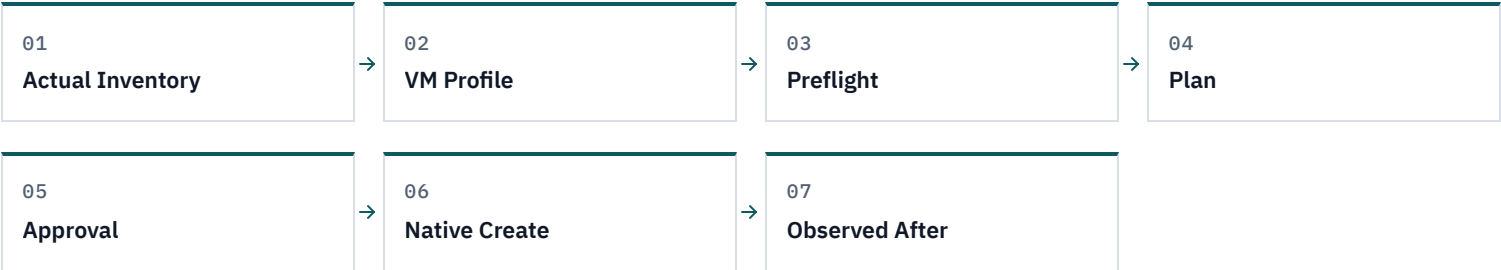
Storage VM은 사용자 확인 운영 구조이며 애플리케이션 세대와 데이터 수명주기를 분리합니다.

실제 상태를 먼저 확인하고, 승인된 계획만 Proxmox에 반영합니다.

현재 문제에는 Terraform·Ansible을 active dependency로 유지하는 것보다 Proxmox API 기반 Native Create가 더 직접적이었습니다. 도구를 포기한 것이 아니라 실제 상태와 승인 경계를 직접 제어하기 위해 다시 선택했습니다.

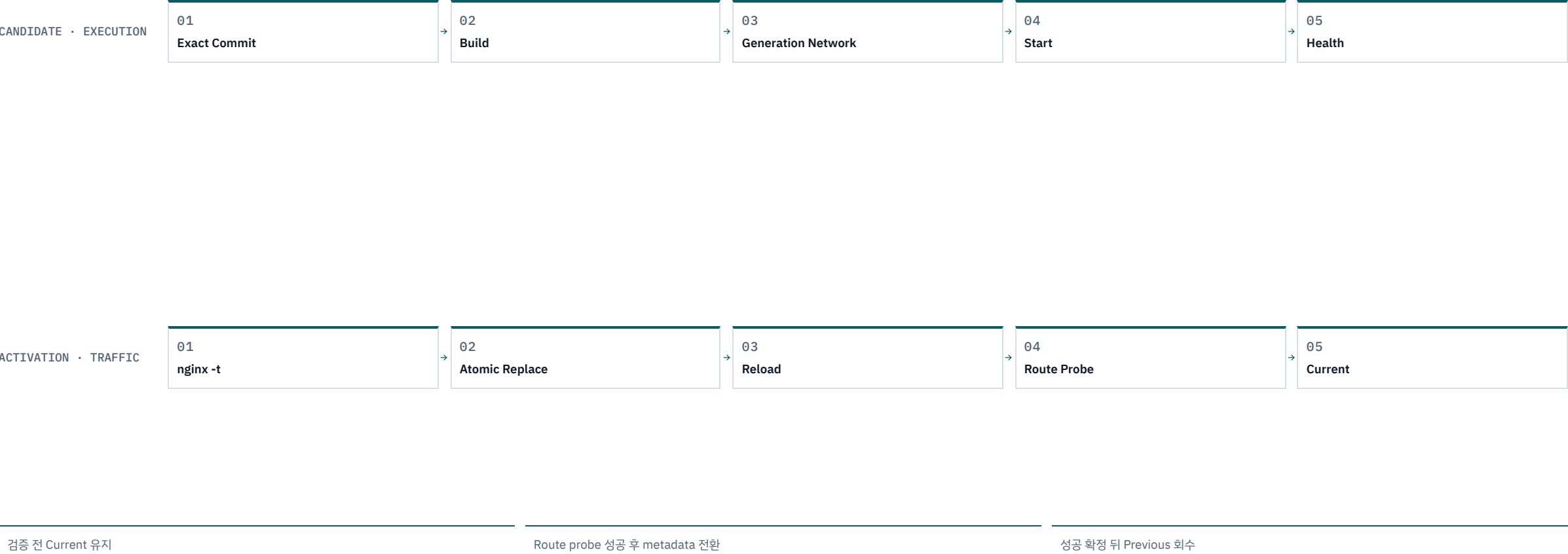
Inventory를 기준으로 Profile과 template·IP·network·storage 조건을 생성 전에 확인합니다.

전체 VM LIFECYCLE · 현재 범위 아님



기존 서비스를 먼저 멈추는 대신, 후보 세대를 분리해 검증합니다.

이전 MVP의 stop-old-then-run-new 방식은 새 배포 실패가 기존 서비스까지 영향을 줄 수 있었습니다. 현재는 고정한 commit과 설정 snapshot으로 Candidate를 만들고 Current와 분리해 검증합니다.



실패 원인을 넓게 지우지 않고, 확인한 후보만 정리합니다.



서로 다른 수명주기

 **Runtime Generation**
검증된 후보 단위로 교체

≠

 **PostgreSQL Data**
별도 Storage VM · Operational

배포 요청부터 세대와 현재 상태까지 하나의 실행 기록으로 추적합니다.

HEIMDALL

Preview Deployment Console

PREVIEW SERVICES API base: http://127.0.0.1:18180

Projects

Deployments

Settings

PROJECTS

1

NO LIVE PREVIEW

0

AUTO DEPLOY

1

DEPLOYING

0

FAILING

0

Projects

Table-first project list. Select a row to inspect and operate on it.

Register project

PROJECT	PROVIDER	BRANCH	STATUS	PREVIEW TARGET	LAST DEPLOY	AUTO	ACTIONS
gjallr gjallr	github	main	SUCCESS	http://127.0.0.1:18002 real preview active	2026. 6. 17. 오후 11:38:05	On	Deploy preview Toggle auto

Project Detail

gjallr

REAL PREVIEW ACTIVE

gjallr

https://github.co...

Deploy preview

Edit settings

Register/reuse webhook

Disable auto

Delete

Tracked branch

main

Services

1 service

Container port

8000

Health check

/health

Current commit

a6e187309140cb2dbef8734e3536c39d8689d93c

Preview target

http://127.0.0.1:18002

Webhook

NOT REGISTERED

Managed PostgreSQL

ACTIVE

Purge

Releases / Rollback

Image rollback changes preview containers only; managed PostgreSQL data is not restored.

This project has a managed PostgreSQL binding. Rollback does not restore database data.

COMMIT	IMAGE TAG	STATUS	CREATED	ACTION
a6e1873	app PREVIEW ENTRY AVAILABLE heimdall/gjallr:a6e1873 http://127.0.0.1:18002	CURRENT	2026. 6. 17. 오후 11:38:05	Rollback image

저장소 검증

고정한 commit, 배포 세대와 현재 상태를 확인하는 Heimdall 운영 화면

- 01 Exact commit
요청 시점의 commit과 설정 snapshot을 고정
- 02 Generation state
Candidate의 실행과 health 상태를 추적
- 03 Current route
검증을 통과해 운영 경로가 된 세대를 확인

도구는 달라도, 입력과 실행 사이에 경계를 두는 판단은 반복됐습니다.

Argus KRX

장중 · 업데이트 3월 25일 오후 01:56 · 관심종목

티커/종목명/키워드 검색

대시보드 · AI 인사이트 · 파생·수급 · 시장 뉴스 · 매크로 캘린더

STATUS SHELL

공통 상태 헤더

장중 소스 2/3 업데이트 3월 25일 오후 01:56

장 상태, 소스 커버리지, 속도만 짧게 확인하는 공통 셸입니다. 메인 해석은 각 탭 첫 색션에서 읽습니다.
3/3개 소스 반영, 일부 지연이 있습니다.

뉴스 해석 · 정상 · 시장 이벤트 · 정상 · 파생 신호 · 부분

속보 한국은행, 기준금리 동결 기조 재확인

은행주와 내수주 상대 강도 변화 가능성이 커졌습니다.

한국 증시 · 연평 탭 · 시장 뉴스 · 3월 24일 오후 01:56 · Argus KRX Desk

관련 탭 보기

OVERVIEW DESK

대시보드

3월 24일 오후 01:55

지금 뭐가 중요한지 60초 안에 먼저 정리하는 cockpit입니다.

지금 뭐가 중요하지?

현물과 파생 모두 아직 한쪽으로 강하게 기운 신호는 아닙니다.

핵심 드라이버 3개

30초 안에 먼저 볼 이유

DRIVER 1

개별 종목 영향이 우선이어서
시장 전체보다 종목 반응을 먼저
확인해야 합니다.

DRIVER 2

[기재정정]사업보고서
(2023.12)

DRIVER 3

해외 변수지만 원화, 위험선호,
한국 증시 수급에 연동될 가능
성이 큼니다.

GATEWAY

오늘 먼저 볼 탭

파생·수급

LOW

최근 세션 파생 지표 기준으로 방향성은 증립, 겹
해석은 겹 중립, 변동성은 변동성 안정로 해석했
습니다. (신뢰도 low) 본 해석은 정보 제공 목적이며
투자 성과를 보장하지 않습니다.

신뢰도 · 업데이트 대기

시장 뉴스

67건

[기재정정]사업보고서 (2023.12) 중심으로 KR 이
슈를 먼저 확인할 수 있습니다.

핵심 카드 · 3월 24일 오후 01:55

저장소 검증

Argus · Implemented · provider boundary와 normalized snapshot

K-Le-PaaS에서는 자연어 입력을 CommandPlan으로 정규화했습니다. Argus에서는 서로 다른 공급자를 adapter로 수집하고 normalized snapshot을 거쳐 판단 로직이 원천 차이를 직접 떠안지 않게 했습니다.

Previous · K-Le-PaaS

Input → CommandPlan → Execution

2인 팀 · 검증된 개인 기여 분리

Implemented · Argus

Provider → Snapshot → Judgement

공급자 경계와 정규화된 상태

도메인은 달라도 입력을 바로 핵심 실행 로직에 연결하지 않는 판단은 같았습니다.

YUNHO CHO · PLATFORM ENGINEER

13 / 14

운영 문제를 발견하고, 자동화한 뒤, 실패 경계를 다시 설계합니다.

완성된 구조만 보여주기보다 첫 판단이 왜 바뀌었는지, 실패했을 때 무엇을 지키려 했는지, 아직 구현하지 않은 범위까지 설명할 수 있는 엔지니어가 되고자 합니다.

- 01 실행 전에 실제 상태와 계획을 확인합니다.
- 02 변경 이유가 다르면 책임과 실패 경계를 분리합니다.
- 03 불확실한 상태는 자동 삭제보다 보존을 선택합니다.

현재 범위

Implemented Heimdall Verified generation promotion	Implemented Gjallar Approved native create
Previous · 2인 팀 K-Le-PaaS Verified personal CommandPlan contribution	Implemented Argus Provider boundary & normalized snapshot

Planned External Exposure Architecture

CORE Proxmox · Docker · Kubernetes · FastAPI · PostgreSQL · TypeScript	EDUCATION 광운대학교 전자통신공학과 · 2026.02 졸업	CERTIFICATION 정보처리기사 · 리눅스마스터 2급
---	---	-------------------------------------

Email
code.penguin.yoon@gmail.com

GitHub
github.com/CodingPenguin-yoon

Web
yoonman.page